

AI-Powered Fire Spread Prediction and Containment System

System Requirements Specification

Panic! At The Kernel
Client: EPI-USE Africa

May 2026



FIRE **AWAY**

AI-POWERED FIRE SPREAD PREDICTION AND CONTAINMENT SYSTEM



Full Name	Student Number
Megan Lai	u23608821
Inge Keyser	u23544563
Janri du Toit	u23632730
Ryan Lynn	u23541912
Marco de Wit	u22811975

1 Introduction

AI-Powered Fire Spread Prediction and Containment System, is designed to address the critical business need for real-time situational awareness and rapid emergency response by utilizing AI-driven simulations and multi-source verification to mitigate the impact of fire disasters. Developed according to IEEE 29148 and ISO 22320 standards, the project's scope encompasses an end-to-end ecosystem that includes secure authentication and role-based access control for users, firefighters, and administrators. The core functionality features a public fire reporting interface with automated verification via NASA FIRMS satellite data, which triggers a sophisticated AI simulation engine to produce color-coded spread predictions based on weather and terrain variables. Furthermore, the system includes a live mapping interface with proximity-based push notifications for registered users, specialized tactical tools for firefighters to log containment lines and view AI recommendations, and an offline map cache to ensure operational continuity in areas with limited connectivity.

2 User Characteristics and User Stories

2.1 User Characteristics

The system supports four user classes, each with distinct access levels and responsibilities.

- **Anonymous Visitor:** Can report fires and view the live map without an account.
- **Registered User:** Receives push notifications and access to AI fire-spread overlays.
- **Firefighter:** Has access to tactical tools, containment line logging, and live field coordinates.
- **Administrator:** Manages user roles, reviews security logs, and maintains the incident archive.

2.2 User Stories

2.2.1 E1: Authentication and Access Control

- As an anonymous visitor, I want to register with an email and password so that I can receive push alerts.
- As a registered user, I want to log in so that I can access my role-based dashboard.
- As an administrator, I want to approve or revoke user roles so that only authorised personnel can access operational tools.

2.2.2 E2: Fire Reporting and Verification

- As any user, I want to report a fire without logging in so that I can alert others immediately.
- As the system, I want to auto-verify reports so that false alerts are minimised.
- As a firefighter or administrator, I want to manually verify or reject pending reports so that unconfirmed incidents receive appropriate review.

2.2.3 E3: AI Fire-Spread Simulation

- As the system, I want to generate a fire-spread prediction upon verification so that users and responders can assess projected risk immediately.
- As the system, I want to re-run the simulation when a containment line is logged so that the prediction reflects current field actions.

2.2.4 E4: Live Fire Map

- As any user, I want to view the live fire map without logging in so that I can see active fires near me.
- As a registered user, I want to see the AI spread-prediction overlay so that I can assess risk in my area.

2.2.5 E5: Notification and Alert System

- As a registered user, I want to receive a push notification when a fire is verified near me so that I can take immediate action.

2.2.6 E6: Firefighter Tactical Tools

- As a firefighter, I want a tactical dashboard with AI recommendations so that I can make informed decisions in the field.
- As a firefighter, I want to draw containment lines on the map so that the AI simulation reflects my ground actions.

2.2.7 E7: Offline Map Cache

- As a firefighter, I want to view cached map data when offline so that I retain situational awareness without connectivity.

2.2.8 E8: Administration Panel

- As an administrator, I want a management panel to approve role requests and revoke access so that system permissions remain controlled.

3 Functional Requirements

3.1 E1: Authentication and Access Control

- R1.1 The home screen shall present three options: Log In, Register, and Continue without Account.
- R1.2 New registrations shall automatically be assigned the Registered User role.
- R1.3 The Administrator role shall require approval by an existing Administrator. The first administrator shall be provisioned by system seed at deployment time.
- R1.4 Password reset shall be performed via a time-limited email link. The link shall expire after 15 minutes and be single-use.
- R1.5 All role and permission checks shall be enforced server-side at the API layer.
- R1.6 JWT tokens shall be used for all authenticated API calls. Access tokens shall be short-lived (e.g., 15 minutes) and paired with refresh tokens.
- R1.7 All client-server communication shall use HTTPS/TLS.
- R1.8 All authentication events and role changes shall be written to the audit log.
- R1.9 Tokens shall be invalidated immediately on logout, role change, or account suspension. Because pure JWTs cannot be instantly revoked without a denylist, the system relies on short TTL and refresh token rotation.

- R1.10 Two-factor authentication (TOTP) shall be available for all user roles. Users may enable 2FA after login; a setup endpoint returns an otpauth:// URI.
- R1.11 The system shall enforce rate limiting on login attempts. Limit: 5 failed attempts per 15 minutes per account. After 10 consecutive failures, the account shall be locked for 30 minutes.

3.2 E2: Fire Reporting and Verification

- R2.1 An unregistered user shall be able to submit a fire report.
- R2.2 The fire reporting map shall be accessible from the home screen without login.
- R2.3 Required report inputs shall be: map pin location and estimated fire size.
- R2.4 Optional report fields shall include: photo attachment and free-text description.
- R2.5 The system shall generate a unique reference number for every report.
- R2.6 All reports shall enter Pending status; no alerts shall be triggered until Verified.
- R2.7 GPS auto-fill shall be offered; manual map pin placement shall always be available.
- R2.8 Duplicate reports at the same location shall be automatically linked to one incident.
- R2.9 Auto-verification shall trigger on: (a) two independent same-location reports, or (b) a NASA FIRMS satellite match.
- R2.10 Manual verification and rejection shall be available to Firefighter and Administrator roles.
- R2.11 Rejection shall require a mandatory reason field.
- R2.12 Registered reporters shall be notified on verification or rejection of their report.
- R2.13 All reports shall be stored in the database for auditing purposes.
- R2.14 Report status values shall be: Pending, Verified, and Rejected.

3.3 E3: AI Fire-Spread Simulation

- R3.1 The simulation shall start automatically when a fire report is set to Verified.
- R3.2 The simulation engine shall ingest: wind speed and direction, temperature, humidity, precipitation, and vegetation dryness index.
- R3.3 The simulation engine shall account for terrain slope and vegetation type.
- R3.4 Output shall be a colour-coded spread probability grid for 1 h, 3 h, 6 h, and 24 h horizons. Grid resolution: 10 m × 10 m cells. Probability scale: 5 discrete bands (0–20%, 21–40%, 41–60%, 61–80%, 81–100%).
- R3.5 The initial prediction shall be delivered within 5 seconds of verification under normal load (one active fire, weather cache warm). If the simulation exceeds 10 seconds, the engine shall fall back to the last valid prediction (if any) and mark it as “degraded”; otherwise, it shall return a wind-biased elliptical buffer around the ignition point.
- R3.6 The engine shall re-run automatically when new weather data is received or a containment line is logged.

- R3.7 If a data source is unavailable, the engine shall use last known values and shall never produce a blank map. Predictions generated using data older than 1 hour shall include a data_{quality}flagindicatingwhichinputs
- R3.8 The simulation engine shall be versioned. Each prediction shall record the model version (e.g., “wind.ca.v1.0”). Model version and input timestamps shall be stored in the audit log to enable post-hoc reproducibility.
- R3.9 The baseline implementation shall be a wind-biased cellular automaton (CA) model. Fire spreads from ignition cell to neighbouring cells with probabilities proportional to wind speed and direction, reduced by humidity and vegetation dryness. A convolutional neural network (CNN) with LSTM layers for temporal weather trends is a stretch goal, not required for the first demonstrator.
- R3.10 The system shall meet a minimum accuracy target evaluated against historical fire incidents. For a held-out test set of at least 20 historical fires (ignition point, weather, and observed final perimeter), the 6-hour predicted spread polygon shall achieve an Intersection-over-Union (IoU) of at least 0.30 against the observed burned area. This metric shall be recomputed after each major model update.

3.4 E4: Live Fire Map

- R4.1 The map shall load without login; verified fire markers shall be visible to all users.
- R4.2 Spread prediction overlays shall be visible to Registered Users and above.
- R4.3 Tapping a fire marker shall display: status, report time, and estimated fire size.
- R4.4 The map shall auto-refresh on new data with no manual reload required.
- R4.5 The map shall be responsive on mobile, tablet, and desktop.
- R4.6 Map tiles shall be cached locally to support basic offline viewing.

3.5 E5: Notification and Alert System

- R5.1 Push alerts shall be dispatched within seconds of a fire report reaching Verified status.
- R5.2 Alerts shall be delivered as push notifications to registered users.
- R5.3 Notifications shall appear when the app is closed or running in the background.
- R5.4 Topic-based grid-zone messaging shall be used for simultaneous large-scale delivery.
- R5.5 The message broker shall retain and retry failed deliveries.
- R5.6 Notification content shall include: fire location, distance from user, severity, and a map deep-link.
- R5.7 Every dispatched alert shall be logged with a timestamp, zone ID, and distance.

3.6 E6: Firefighter Tactical Tools

- R6.1 Firefighters shall have access to a tactical dashboard displaying a live fire map and AI recommendations.
- R6.2 The dashboard shall show: fire direction, high-risk areas, wind speed and direction, and suggested containment locations.
- R6.3 The firefighter’s GPS position shall be shown as a live icon on the map.

- R6.4 Firefighters shall be able to draw containment lines directly on the map.
- R6.5 Logged containment lines shall trigger an automatic simulation re-run.
- R6.6 Every logged action shall be stored with GPS coordinates and a timestamp.
- R6.7 The interface shall be optimised for field use with large tap targets and high contrast.

3.7 E7: Offline Map Cache

- R7.1 The app shall cache map tiles and the most recent spread prediction while online.
- R7.2 An offline indicator shall be displayed when the device has no connectivity.
- R7.3 Cached map data and the most recent prediction shall remain viewable while offline.

3.8 E8: Administration Panel

- R8.1 The admin panel shall support: approving and rejecting role requests, searching users, and revoking access.
- R8.2 The admin panel shall enforce its own server-side role check.
- R8.3 Closed fire incidents shall be moved to a searchable archive automatically.
- R8.4 All role changes and access revocations shall be written to the audit log.

4 Use Cases

4.1 Authentication & Access Control

UC-1.1 - User Registration Actor: [3]

Anonymous visitor | Trigger: User taps Register

1. User enters email and password (validated inline).
2. System sends verification email.
3. User confirms email – > Registered User role assigned; JWT issued.

UC-1.2 - User Login Actor: [2]

Registered User / Firefighter / Admin | Trigger: User taps Log In | Pre: Active account exists

1. User enters email and password.
2. System validates credentials server-side.
3. JWT issued; user directed to role-appropriate home view.

UC-1.3 - Admin Role Approval / Revocation Actor: [5]

Admin | Trigger: Role request received or account action required | Pre: Admin role

1. Admin opens the user management panel; views pending requests or searches by name/email.
2. Admin approves or rejects the role request (admin requests).
3. Admin can revoke account access.

4.2 Fire Reporting & Verification

UC-2.1 - Fire Report Submission Actor: [4]

Any user | Trigger: User taps somewhere on the map to report a fire | Pre: None

1. The reporting screen opens with a map view.
2. The system attempts to auto-fill the device's GPS location as a pin.
3. The user can confirm the auto-filled pin or adjust it by dragging to the correct location.
4. The user enters the estimated fire size (mandatory), and optionally adds a photo and a description.
5. User taps Submit.
6. System assigns a unique reference number; report status set to Pending.
7. Auto-verification check runs immediately (UC-2.2 [4.2]).

UC-2.2 - Auto-Verification: [4]

Actor: System | Trigger: New Pending report | Pre: Pending report exists

1. System checks:

- two or more independent reports within a configurable radius, or
 - NASA FIRMS hotspot match.
2. If either condition met, – > status set to Verified; AI simulation triggered (UC-3.1 [4.3]); push alerts dispatched (UC-5.1 [4.5]).

UC-2.3 - Manual Verification / Rejection: [4]

Actor: Firefighter or Admin | Trigger: Pending report requires review | Pre: Firefighter or Admin role

1. The reviewer selects a Pending report and examines the details and photos.
2. Reviewer taps Verify or Reject (rejection requires a mandatory reason).
3. Verify – > same downstream actions as UC-2.2 [4.2] steps 2. Reject – > status set to Rejected; reporter notified with reason.

4.3 AI Fire Spread Simulation

UC-3.1 - Generate Fire Spread Prediction:

Actor: System | Trigger: Fire report Verified | Pre: Verified fire location exists

1. Engine fetches wind speed/direction, temperature, humidity, precipitation, vegetation dryness index.
2. Engine loads terrain slope and vegetation type.
3. Simulation runs; colour-coded probability grid produced for 1 h, 3 h, 6 h, and 24 h horizons.
4. Result published to live map within seconds of verification.

UC-3.2 - Re-Run After Containment Line:

Actor: System | Trigger: Firefighter logs containment line (UC-6.2) | Pre: Active verified fire

1. System receives containment line coordinates.
2. Engine re-runs simulation with line as a fire barrier.
3. Updated prediction published within seconds.

UC-3.3 - Re-Run on Weather-Update:

Actor: System | Trigger: Firefighter logs containment line (UC-6.2) | Pre: Active verified fire

1. The system detects a weather data update that affects the active fire zone
2. The simulation engine is automatically re-run with the new weather data.
3. The new prediction is published to the live map and logged with a timestamp and weather-update flag.

4.4 Live Fire Map

UC-4.1 - View Live Fire Map: [6]

Actor: All roles | Trigger: User opens app | Pre: None

1. Map loads immediately without login.
2. Verified fire markers displayed; Pending fires not shown publicly.
3. Tapping a marker shows: status, report time, and estimated size.
4. Registered users also see the spread overlay and at-risk roads/settlements.
5. Map auto-refreshes on new data; no manual reload required.

4.5 Notification & Alert System

UC-5.1 - Dispatch Proximity Push Alerts:

Actor: System | Trigger: Fire status changes to Verified | Pre: Registered users exist in alert zone

1. System identifies at-risk users via grid-based topic messaging.
2. Push notification dispatched via FCM within 5 s of Verified status.
3. Content: fire location, distance from user, severity, map deep-link.
4. User taps notification – > app opens directly to fire on map.
5. Alert logged with timestamp, zone ID, and distance.

4.6 Firefighter Tactical Tools

UC-6.1 - Tactical Dashboard:

Actor: Firefighter | Trigger: App opened during active incident | Pre: Firefighter role; active verified fire

1. Tactical dashboard opens with live fire map and spread overlay.
2. AI panel shows: fire direction, high-risk areas, wind speed/direction, and suggested containment spots.
3. Firefighters' live GPS position is shown as an icon on the map.

UC-6.2 - Log Containment Line:

Actor: Firefighter / any user with prior connectivity | Trigger: Device loses connectivity | Pre: Data previously cached

1. Firefighter selects the “Draw Containment Line” tool.
2. Firefighter draws a line on the map.
3. System saves the line as a set of GPS coordinates with a timestamp.
4. Simulation re-run is triggered.
5. Updated spread prediction (incorporating the line as a barrier) is displayed on the map .

4.7 Offline Map Cache

UC-7.1 - View Map While Offline:

Actor: Firefighter / any user with prior connectivity | Trigger: Device loses connectivity | Pre: Data previously cached

1. App detects connectivity loss; 'Offline' banner displayed.
2. Cached map tiles and the most recent spread prediction remain visible.
3. On reconnection, the app silently refreshes to live data.

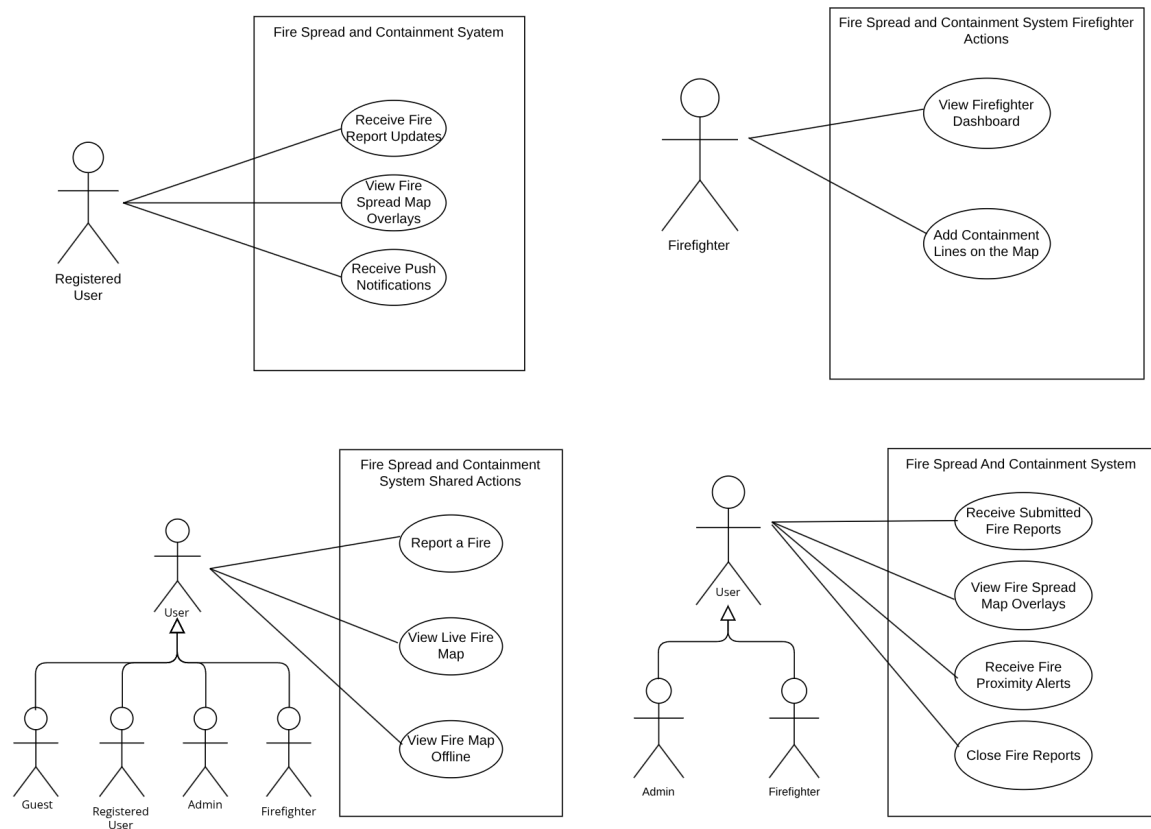


Figure 1: Role Based Access Control

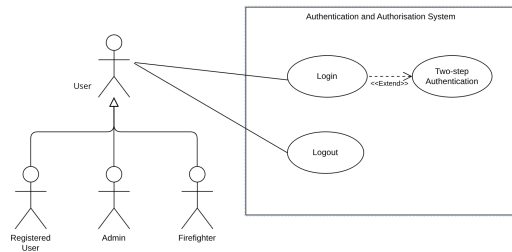


Figure 2: Authentication and Authorisation System

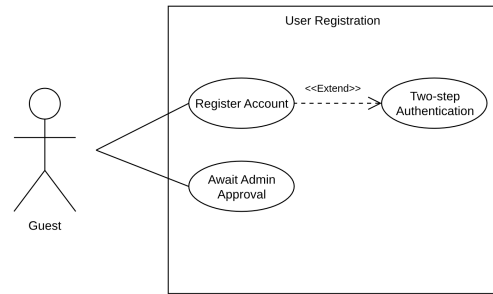


Figure 3: Registration System

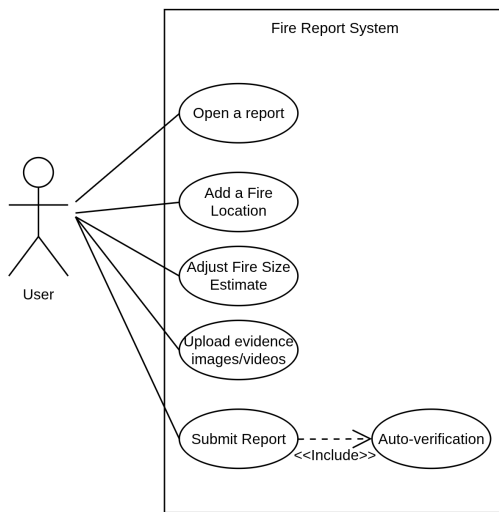


Figure 4: Reporting a Fire System

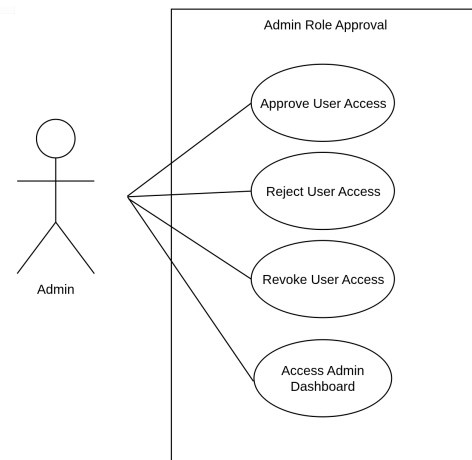


Figure 5: Admin Approval System

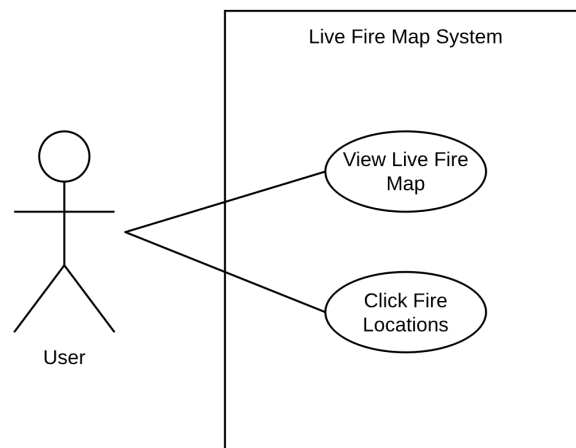


Figure 6: Live Fire Map System

5 API Service Contracts

5.1 Authentication & Access Control Service

Description: Manages user registration, login, and secure session management using JWT tokens.

Inputs:

- **email** (string) – User’s unique email address.
- **password** (string) – User’s secure password.

Outputs:

- **access_token** (JWT) – Token used for all subsequent authenticated API calls.
- **role** (string) – The assigned user role (e.g., Registered User, Firefighter, Admin).

Usage / Interaction Rules:

- Accessed via HTTPS/TLS.
- Tokens are invalidated immediately upon logout or role change.
- Authentication events are written to an audit log.

5.2 Fire Reporting & Verification Service

Description: Processes new fire reports from users and manages the verification status of incidents.

Inputs:

- **location** (coordinates) – GPS pin location of the fire.
- **timestamp** (DateTime) – Time the report is submitted.
- **fire_size** (string/int) – Estimated size of the fire.
- **photo** (file, optional) – Image attachment of the fire.
- **description** (string, optional) – Free-text details provided by the reporter.

Outputs:

- **reference_number** (string) – TA unique identifier for the report.
- **is_duplicate** (boolean) – Returns true if a report is a duplicate.
- **status** (string) – Initial status set to "Pending".

Usage / Interaction Rules:

- Can be called by anonymous (unregistered) users.
- Duplicate reports at the same location are automatically linked to one incident.

5.3 AI Fire Spread Simulation Engine

Description: Calculates predicted fire spread horizons based on environmental data and terrain.

Inputs:

- **fire_location** (coordinates) – The verified origin of the fire.
- **weather_data** (object) – Includes wind speed, direction, temperature, and humidity.
- **terrain_data** (object) – Includes slope and vegetation dryness index.
- **barriers** (array, optional) – Coordinates of logged containment lines.

Outputs:

- **spread_grid** (JSON/GeoJSON) – Colourcoded probability grid for 1h, 3h, 6h, and 24h horizons.

Usage / Interaction Rules:

- Triggered automatically once a fire status changes to "Verified".
- Re-runs automatically when new weather data or containment lines are received.

5.4 Live Fire Map Service

Description: Provides real-time data for fire markers and spread overlays to the client application.

Inputs:

- **user_role** (string) – Used to determine if the requester can see the spread overlay.
- **map_bounds** (coordinates) – The current viewport to filter visible markers.

Outputs:

- **active_fires** (list) – Array of verified fire markers with status, time, and size.
- **overlay_data** (object) – Prediction grid (visible to Registered Users only).

Usage / Interaction Rules:

- Service must support auto-refresh to push new data without manual reloads.
- Pending fires are filtered out and not shown to the public.

5.5 Proximity Alert Service

Description: Dispatches push notifications to users within a specific risk zone of a verified fire.

Inputs:

- **fire_id** (string) – Reference to the verified fire incident.
- **zone_id** (string) – The grid zone identified as at-risk.

Outputs:

- **delivery_status** (boolean) – Confirmation that the message was sent to the broker.

Usage / Interaction Rules:

- Must dispatch notifications via Firebase Cloud Messaging (FCM) within 5 seconds of verification.
- Messages must include a deep-link to the specific fire on the map.

5.6 Firefighter Tactical Service

Description: Handles specialized inputs from field personnel, such as GPS tracking and containment logging.

Inputs:

- **firefighter_id** (string) – Unique ID of the responder.
- **current_gps** (coordinates) – Real-time location of the firefighter.
- **containment_line** (array) – Set of GPS coordinates representing a firebreak.

Outputs:

- **sync_status** (string) – Confirmation of data storage and simulation trigger status.

Usage / Interaction Rules:

- Strictly restricted to users with the "Firefighter" role.
- Every logged action must be stored with a high-precision timestamp for auditing.

6 Domain Model

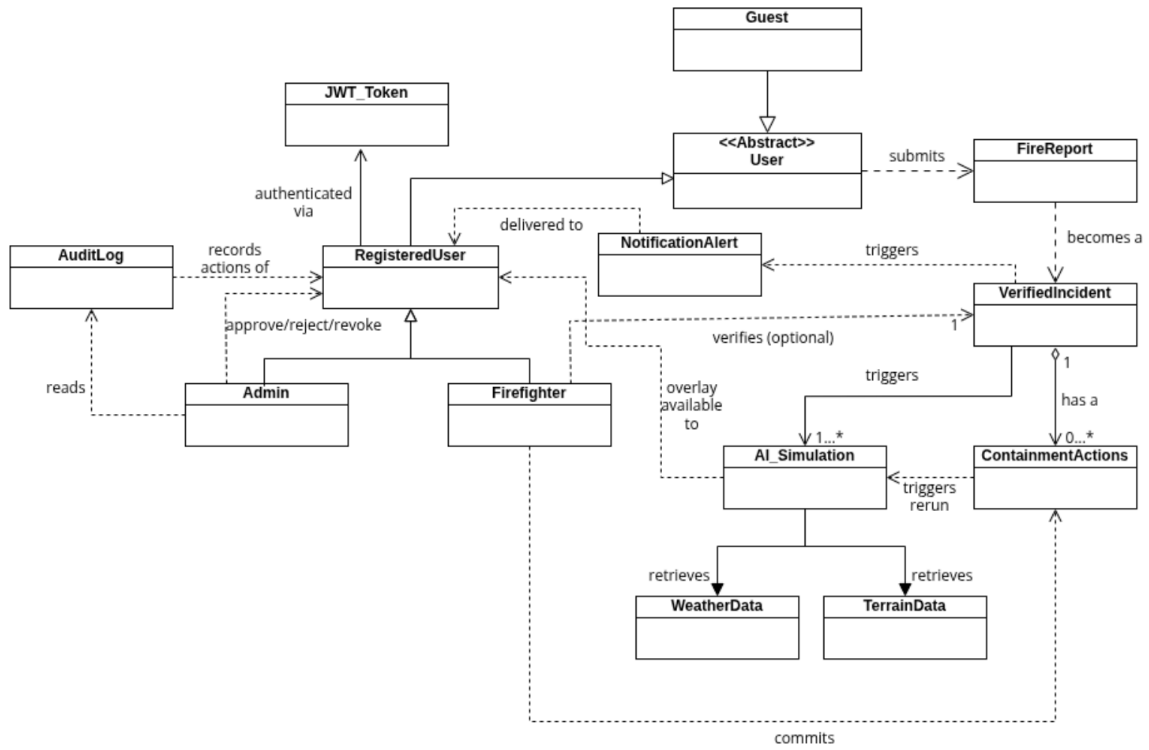


Figure 7: AI-Powered Fire Spread Prediction and Containment System Domain Model

7 Architectural Requirements

7.1 Architecture Diagram

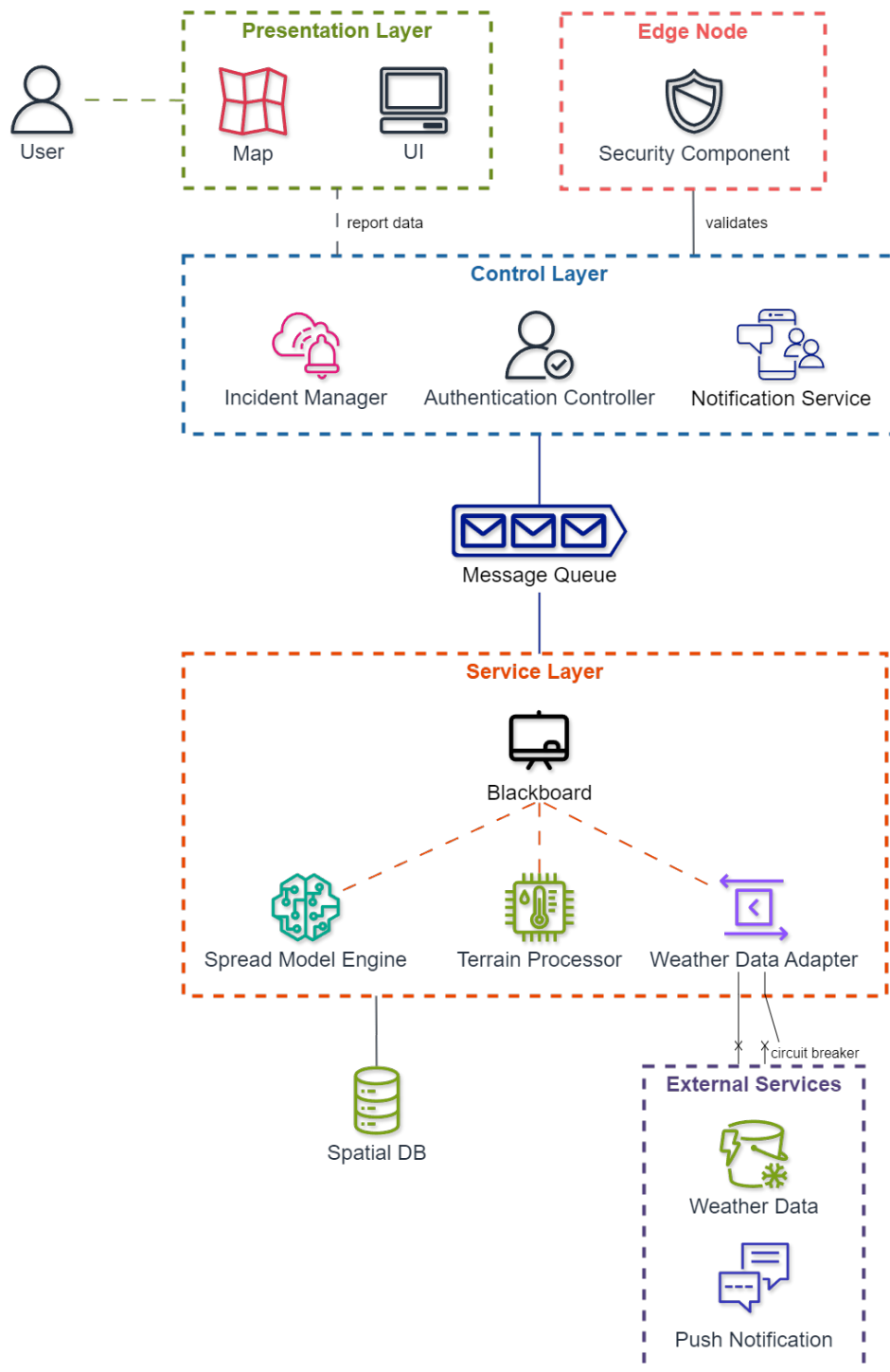


Figure 8: Architecture Diagram

7.2 Architectural Patterns

7.3 Quality Requirements

7.3.1 Scalability

- Support horizontal scaling to handle a minimum of 1000 concurrent users and fire reports, ensuring smooth simulation and map updates without downtime.

7.3.2 Security

- End-to-end encryption is used for user data, role-based access control (RBAC) for firefighters and admins, and secure API authentication for external data sources.

7.3.3 Availability

- Ensures real-time updates and interactive maps function optimally across devices, including low-spec mobile devices and areas with intermittent connectivity.

7.3.4 Performance

- Optimized simulation engine and data processing for rapid fire spread predictions, with smart caching for frequently accessed environmental and historical data.

7.3.5 Reliability

- Maintains accurate fire spread predictions, consistent logging of user and firefighter actions, and robust handling of incomplete or delayed data inputs.

7.3.6 Extensibility

- Allows integration of additional environmental datasets, satellite sources, or fire risk models without major system redesign.

8 Technology Requirements

Layer	Technology	
Frontend	React, Next.js, DaisyUI	  
Backend	Python, FastAPI	 
Mapping	Mapbox GL	 mapbox
AI/ML	PyTorch	
Database	PostgreSQL, PostGIS, SQLAlchemy	 
Cache	Valkey	 Valkey
Notifications	AWS SNS	
Versioning	Git, GitFlow	 
Containerization	Docker	
Deployment	TBA	
Testing	Pytest, Playwright	 

8.1 Technology Requirements Choices

8.1.1 Frontend

- React
- Next.js
- DaisyUI

React's concurrent rendering handles simultaneous live map updates without bottlenecks. Next.js SSR ensures WCAG 2.1 compliance as screen readers require fully rendered HTML. DaisyUI handles the structural baseline and the state logic of UI elements, allowing to treat the frontend as a modular, plug-and-play system.

8.1.2 Backend

- Python
- FastAPI

Python has mature support for the full AI/ML stack required (PyTorch, scikit-learn, GeoPandas, GDAL). FastAPI's async model concurrently ingests live ERA5, SAWS, and NASA FIRMS data without blocking, and runs at 21,000+ requests/second (above the 1,000 concurrent user requirement). Pydantic validation meets the 200ms form validation delivery requirement at the API layer.

8.1.3 Mapping

- Mapbox GL

GPU-accelerated WebGL rendering redraws fire spread polygons and overlays in under 16ms. Built-in offline tile packs allow firefighters to use the map without internet. The library is open source and free, with a 50000 map loads/month free tier.

8.1.4 AI/ML

- PyTorch

PyTorch's CNN achieves F1 scores of 0.64 - 0.72 on fire spread benchmarks. ts LSTM layer handles the 30-day vegetation dryness and multi-hour wind forecasts required by the 24 - 72hr risk engine. The autograd engine enables post-event model fine-tuning.

8.1.5 Database

- PostgreSQL
- PostGIS
- SQLAlchemy

PostGIS provides 1 000+ geospatial functions for cross-referencing fire paths with OSM roads and settlements. ACID transactions guarantee complete audit records with rollback capability, ensuring firefighter action logging. GiST spatial indexes enable sub-second proximity queries powering automated infrastructure alerts. SQLAlchemy provides a secure, programmatic database abstraction layer that natively integrates with spatial extensions to handle complex geospatial coordinate data, like fire pins and containment lines, using clean Python code. Additionally, its built-in connection pooling and automated parameter sanitization safeguard your platform against SQL injection while maintaining high performance during sudden concurrent traffic spikes.

8.1.6 Cache

- Valkey

Valkey operates entirely in-memory, delivering sub-millisecond read times. When the AI simulation engine needs wind, humidity, and vegetation data instantly, pulling it from Valkey rather than making repeated external API calls or querying a disk-based SQL database ensures the prediction grid is generated within seconds.

8.1.7 Notifications

- AWS SNS

SNS includes robust, configurable delivery retry policies out of the box. If FCM fails to accept a message, SNS automatically retains the payload and executes an exponential backoff strategy to ensure the alert is eventually delivered, eliminating the need to write custom retry logic in our API.

8.1.8 Versioning

- Git
- GitFlow

Git paired with Gitflow provides isolated, risk-free environments to develop complex features like the AI simulation engine without compromising the stability of the live, safety-critical application. It establishes a dedicated pipeline for rigorous testing and rapid emergency hotfixes, ensuring the platform remains highly reliable and continuously available for field operations.

8.1.9 Containerization

- Docker

Docker containerization locks down all complex geospatial and AI dependencies into an identical environment across development and production, eliminating configuration errors and safeguarding system reliability. It enables lightweight, isolated services that can spin up in milliseconds to horizontally scale under sudden user load during active fire emergencies without risking core system downtime.

8.1.10 Testing

- PyTest
- Playwright

Pytest allows us to rigorously validate the backend API, server-side role checks, and complex simulation edge-cases using reusable fixtures and parameterized data inputs. Meanwhile, Playwright drives actual browsers to emulate real-world field conditions, ensuring responsive map layouts, touch-drawn containment lines, and offline-caching layers work flawlessly for emergency responders.